

Resumo

Este handout apresenta os fundamentos do CSS (*Cascading Style Sheets*), a linguagem responsável pela apresentação visual de páginas web. São abordados a anatomia de uma regra CSS, as formas de incluir estilos em um documento HTML, os principais tipos de seletores, o modelo de cascata e especificidade, o modelo de caixa (*box model*), unidades de medida, propriedades essenciais de texto, cor, dimensões e visibilidade, os sistemas de layout Flexbox e Grid, e uma introdução à responsividade com *media queries*. A parte final mostra como integrar CSS a um projeto Express: como servir arquivos estáticos, organizar múltiplas folhas de estilo e, em casos avançados, gerar CSS dinamicamente a partir do banco de dados. Boas práticas de organização e nomenclatura acompanham cada seção.

gian@uffs.edu.br

Versão: março de 2026

Keywords:

CSS; seletores; especificidade; cascata; box model; Flexbox; Grid; media queries; responsividade; tipografia; cores; boas práticas; Express

Como usar este material: ao longo do handout você encontrará boxes coloridos com diferentes funções. Boxes verdes trazem dicas práticas de desenvolvimento. Boxes amarelos propõem exercícios de fixação. Boxes vermelhos alertam para armadilhas comuns. Boxes azuis apresentam informações contextuais importantes. Todo código deve ser digitado — não copiado — para consolidar o aprendizado.

1. O Que É CSS

CSS (*Cascading Style Sheets*) é a linguagem que controla a **apresentação visual** de elementos HTML: cores, fontes, tamanhos, espaçamentos, bordas, sombras e layout. Enquanto o HTML descreve *o que* o conteúdo é, o CSS descreve *como* ele deve aparecer.

A separação entre HTML e CSS é um dos princípios fundamentais do desenvolvimento web. Uma mesma estrutura HTML pode ter aparências completamente diferentes apenas trocando a folha de estilos — sem tocar no conteúdo. Isso é o princípio SoC (*Separation of Concerns*) aplicado ao *front-end*. O projeto *CSS Zen Garden* (<https://csszengarden.com>) demonstra isso de forma notável: dezenas de designers aplicaram folhas de estilo completamente diferentes ao mesmo HTML, produzindo páginas com aparências radicalmente distintas.

Uma breve história do CSS O CSS foi proposto por Håkon Wium Lie em 1994 e publicado como especificação oficial pelo W3C em 1996 (CSS1). Antes disso, a apresentação visual era controlada com atributos HTML como `<font color="red">` e `<table>` para layout — práticas que misturavam estrutura e apresentação de forma inextricável. O CSS2 (1998) introduziu posicionamento e mídias; o CSS3 (a partir de 2011) adotou um modelo modular com especificações separadas para cores, seletores, Flexbox, Grid, animações e muito mais. Ao contrário do HTML, o CSS3 não tem uma versão monolítica: cada módulo evolui independentemente.

2. Anatomia de uma Regra CSS

O bloco fundamental do CSS é a **regra**. Cada regra é composta por um **seletor** — que indica a quais elementos HTML a regra se aplica — e um **bloco de declarações** entre chaves, contendo pares de **propriedade** e **valor**. A estrutura é sempre a mesma, independentemente do tipo de seletor ou da propriedade usada.

seletor

```
p {  
  color: red;
```

```
  propriedade valor ponto-e-vírgula (separa declarações)  
}
```

Código 1. Anatomia de uma regra CSS rotulada.

bloco de declarações (entre chaves)

```
/* Regra com uma declaração */
h1 {
  color: #1f4e79;
}

/* Regra com múltiplas declarações */
p {
  font-size: 16px;
  line-height: 1.6;
  color: #333333;
  margin-bottom: 1rem;
}

/* Comentários em CSS: entre /* e */ */
```

Código 2. Regras CSS com uma e múltiplas declarações.

Cada parte da regra tem uma função precisa:

- **Seletor:** identifica qual(is) elemento(s) HTML receberá(ão) o estilo. Pode ser um nome de tag (p), uma classe (.destaque), um ID (#cabecalho), ou combinações deles.
- **Propriedade:** o aspecto visual a ser modificado (color, font-size, margin).
- **Valor:** o valor atribuído à propriedade (red, 16px, 1rem).
- **Declaração:** o par propriedade + valor, terminado por ponto-e-vírgula.
- **Bloco de declarações:** conjunto de declarações entre { e }.

O ponto-e-vírgula no final de cada declaração é tecnicamente opcional na última declaração do bloco, mas omiti-lo é uma armadilha clássica: ao adicionar uma nova declaração depois, o erro resultante pode ser difícil de localizar. Por convenção, sempre termine todas as declarações com ponto-e-vírgula.

3. Como Incluir CSS em uma Página HTML

Existem três formas de aplicar CSS a um documento HTML, cada uma com implicações diferentes de manutenção, reutilização e especificidade. A escolha correta entre elas é a primeira decisão de arquitetura de qualquer projeto web.

3.1. Arquivo externo (preferível)

O CSS fica em um arquivo .css separado, referenciado no <head> com <link>. É a forma recomendada para qualquer projeto com mais de uma página:

```
<!-- No <head> do HTML -->
<link rel="stylesheet" href="/css/estilo.css">

<!-- Vantagens:
  - Um arquivo serve múltiplas páginas (DRY)
  - Navegador faz cache: carregado uma única vez
  - Separação total entre estrutura (HTML) e apresentação (CSS)
  - Fácil de versionar, testar e manter -->
```

Código 3. Arquivo externo: link no head e estrutura do .css.

```
/* estilo.css */

/* Qualquer seletor e propriedade CSS é válido aqui */
body {
  font-family: Arial, sans-serif;
  color: #333;
}

h1 { color: #1f4e79; }

.card {
  border: 1px solid #ccc;
  border-radius: 8px;
  padding: 1rem;
```

Código 4. Estrutura interna de um arquivo .css.

```
}
```

O navegador carrega o arquivo CSS uma única vez e o armazena em cache. Em todas as páginas subsequentes do mesmo site, o arquivo é servido diretamente do cache, sem nova requisição ao servidor — o que acelera significativamente o carregamento.

3.2. Tag <style> interna (aceitável)

O CSS fica dentro de uma tag <style> no <head> do próprio HTML. É aceitável para estilos específicos de uma única página que não serão reaproveitados:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Página</title>
  <style>
    /* Regras aqui se aplicam apenas a esta página */
    body { font-family: Arial, sans-serif; }
    h1 { color: #1f4e79; }
    .destaque { background-color: #fff3cd; }
  </style>
</head>
<body>...</body>
</html>
```

Código 5. Tag style interna no head.

Quando usar a tag style Use <style> quando o estilo é genuinamente específico de uma única página e não faz sentido em um arquivo compartilhado — por exemplo, um email HTML, um componente gerado dinamicamente, ou um protótipo rápido. Em projetos reais com múltiplas páginas, prefira sempre o arquivo externo.

3.3. Estilo inline no atributo style (evitar)

O CSS é escrito diretamente no atributo style do elemento HTML. Tem a maior especificidade de todas as formas — o que o torna difícil de sobrescrever. O único uso legítimo em produção é para aplicar estilos calculados dinamicamente via JavaScript, por exemplo ao mover um elemento com `element.style.left = x + 'px'`.

```
<!-- As declarações CSS vão no atributo style, separadas por ; -->
<p style="color: red; font-size: 18px; font-weight: bold;">
  Texto com estilos inline.
</p>

<!-- Evitar em produção:
- Impossível reutilizar (viola DRY)
- Mistura estrutura e apresentação
- Especificidade (1,0,0,0): difícil de sobrescrever com classes
- Dificulta a manutenção mudança de cor requer editar cada elemento -->

<!-- Aceitável para:
- Testes rápidos durante o desenvolvimento
- Estilos calculados e aplicados via JavaScript -->
```

Código 6. CSS inline: sintaxe e quando é aceitável.

Forma	Localização	Reutilizável	Especificidade	Usar quando
Arquivo externo	Arquivo .css separado	Sim (todas as páginas)	Normal	Sempre, em produção
Tag <style>	<head> do HTML	Não (só nesta página)	Normal	Estilos únicos de uma página
Inline (style=)	Atributo do elemento	Não (só este elemento)	Alta (1,0,0,0)	Testes ou JS dinâmico

Tabela 1. Comparação entre as três formas de incluir CSS.

 **Prefira sempre o arquivo externo.** Um arquivo CSS externo é carregado uma única vez pelo navegador e armazenado em cache. Em todas as páginas subsequentes que o referenciam, o navegador usa a versão em cache sem fazer nova requisição — o que acelera o carregamento. Estilos inline têm a maior especificidade possível (discutida na Seção 5), o que os torna difíceis de sobrescrever com regras em arquivo externo.

4. Seletores

Os seletores são o mecanismo pelo qual o CSS identifica quais elementos HTML receberão um estilo. Dominar os seletores é a habilidade mais importante do CSS: um seletor bem escolhido torna o código legível, reutilizável e com especificidade controlada. Um seletor excessivamente específico cria CSS frágil e difícil de sobrescrever.

4.1. Seletores básicos

Os três seletores básicos — elemento, classe e ID — formam a base de qualquer folha de estilo. A regra geral é: use seletores de elemento para estilos globais de tipografia, classes para estilização de componentes, e reserve IDs para JavaScript e âncoras de navegação.

```
/* Seletor de elemento: aplica a TODAS as tags <p> da página */
p { color: #333; }

/* Seletor de classe: aplica a todos com class="destaque" */
.destaque { background-color: #fff3cd; }

/* Seletor de ID: aplica ao elemento com id="cabecalho" */
/* IDs devem ser únicos por página */
#cabecalho { background-color: #1f4e79; color: white; }

/* Seletor universal: aplica a todos os elementos */
* { box-sizing: border-box; }

/* Agrupamento: a mesma regra para múltiplos seletores */
h1, h2, h3 { font-family: 'Inter', sans-serif; color: #1f4e79; }

/* Descendente: <a> dentro de <nav> (qualquer nível de aninhamento) */
nav a { text-decoration: none; color: #2e75b6; }

/* Filho direto: <li> filho imediato de <ul> */
ul > li { list-style-type: disc; }

/* Adjacente: <p> imediatamente após <h2> */
h2 + p { font-size: 1.1rem; }
```

Código 7. Seletores de elemento, classe e ID.

Código 8. Agrupamento e combinação de seletores.

A distinção entre seletor descendente (`nav a`) e filho direto (`nav > a`) é importante: o seletor descendente aplica a todos os `<a>` dentro de `<nav>`, independentemente de quantos níveis de aninhamento existam. O filho direto só aplica aos `<a>` que são filhos imediatos — útil quando há estruturas aninhadas e você não quer que a regra “vaze” para elementos internos.

4.2. Pseudo-classes e pseudo-elementos

Pseudo-classes selecionam elementos em um **estado** específico (como “mouse sobre o elemento” ou “primeiro filho da lista”). Pseudo-elementos selecionam **partes** de um elemento que não existem como tags no HTML, como a primeira linha de um parágrafo ou o conteúdo gerado por CSS antes ou após um elemento.

```
/* Pseudo-classes: estado do elemento */
a:hover { color: #c00; } /* mouse sobre o link */
a:focus { outline: 2px solid #2e75b6; } /* foco via teclado */
a:visited { color: #6a0dad; } /* link já visitado */
```

Código 9. Pseudo-classes e pseudo-elementos comuns.

```
input:disabled { opacity: 0.5; } /* campo desabilitado */
li:first-child { font-weight: bold; } /* primeiro item da lista */
li:last-child { border-bottom: none; }
li:nth-child(2n) { background: #f0f0f0; } /* itens pares */

/* Pseudo-elementos: parte do elemento */
p::first-line { font-variant: small-caps; }
p::first-letter { font-size: 2em; float: left; }
```

A pseudo-classe `:focus` é particularmente importante para acessibilidade: usuários que navegam com o teclado dependem do indicador de foco para saber qual elemento está ativo. Nunca remova o `outline` do `:focus` sem substituí-lo por outro indicador visual — fazer isso torna a página inacessível para usuários com deficiência motora ou visual que usam teclado para navegar.

```
/* Elementos com um atributo específico */
input[required] { border-color: #c00; }

/* Atributo com valor exato */
input[type="email"] { background: url('email-icon.svg') no-repeat; }

/* Atributo cujo valor começa com "https" */
a[href^="https"] { color: green; }

/* Atributo cujo valor termina com ".pdf" */
a[href$=".pdf"]::after { content: " (PDF)"; font-size: 0.8em; }
```

Código 10. Seletor de atributo.

O seletor de atributo é especialmente útil para adicionar indicadores visuais automáticos sem precisar adicionar classes no HTML: `a[href$=".pdf"]::after` adiciona um texto “(PDF)” após qualquer link para um PDF, de forma totalmente automática. O pseudo-elemento `::after` gera conteúdo visual a partir do CSS, sem nenhuma marcação HTML extra.

Para pensar: Dada a seguinte estrutura HTML: `<nav><ul><li><a href="#">»Início</a></li></ul></nav>`, escreva quatro seletores diferentes que selecionem o link `<a>`: um seletor de elemento, um seletor descendente, um seletor filho direto e um seletor de atributo. Calcule a especificidade de cada um (veja a Seção 5).

5. Cascata e Especificidade

O “C” de CSS significa *Cascading* (cascata). O nome descreve o mecanismo central da linguagem: quando duas ou mais regras se aplicam ao mesmo elemento e definem a mesma propriedade, o CSS não simplesmente usa a última regra — ele aplica um algoritmo determinístico que considera a **origem**, a **especificidade** e a **ordem** das regras para decidir qual prevalece. Compreender esse algoritmo é essencial para depurar CSS e evitar o uso desnecessário de `!important`.

5.1. Ordem de especificidade

```
/* Especificidade crescente a de baixo vence se houver conflito */

p { color: black; } /* (0,0,1) 1 elemento */
.texto { color: blue; } /* (0,1,0) 1 classe */
#principal { color: green; } /* (1,0,0) 1 ID */

/* Inline style: (1,0,0,0) vence qualquer seletor de folha */
/* <p style="color: red"> */

/* !important: vence tudo use apenas em último recurso */
p { color: purple !important; }
```

Código 11. Especificidade: quem vence quando há conflito.

A especificidade é calculada como quatro números (a, b, c, d), onde cada componente conta um tipo diferente de seletor. Dois seletores são comparados da esquerda

para a direita: o primeiro número maior vence. Se dois seletores tiverem a mesma especificidade, o que aparecer por último no CSS vence (ordem da cascata).

Peso	Origem	Exemplo	Valor
a	Estilos inline	style="..."	(1,0,0,0)
b	IDs	#id	(0,1,0,0)
c	Classes, pseudo-classes, atributos	.classe, :hover, [type]	(0,0,1,0)
d	Elementos e pseudo-elementos	p, h1, ::before	(0,0,0,1)

Tabela 2. Como calcular a especificidade de um seletor.

```
p          /* (0,0,0,1) */
.classe    /* (0,0,1,0) */
p.classe   /* (0,0,1,1) elemento + classe */
#id .classe /* (0,1,1,0) ID + classe */
nav ul li a /* (0,0,0,4) 4 elementos */
```

Código 12. Calculando especificidade de seletores compostos.

⚠ Evite usar !important e IDs como seletores CSS. !important quebra o fluxo natural da cascata e torna o CSS difícil de depurar. Se você está usando !important com frequência, é sinal de que a arquitetura de seletores precisa ser revisada. Da mesma forma, usar IDs como seletores CSS (#cabecalho { ... }) cria especificidade muito alta, que se torna difícil de sobrescrever. Prefira classes para estilização — reserve IDs para JavaScript e âncoras.

5.2. Herança e valor inicial

Algumas propriedades CSS são **herdadas**: um elemento filho automaticamente recebe o valor definido no pai se não tiver regra própria. Isso é uma conveniência poderosa — ao definir font-family no body, todos os elementos da página herdam essa fonte sem precisar de regras individuais. Outras propriedades não são herdadas por padrão, pois herdar border ou padding de cada ancestral produziria resultados indesejados.

```
/* color e font-family são herdadas filhos recebem automaticamente */
body {
  font-family: 'Inter', Arial, sans-serif;
  color: #333;
  /* Todos os elementos dentro de <body> herdarão esses valores */
}

/* border e padding NÃO são herdadas */
/* Cada elemento define as suas próprias */

/* Forçar herança explicitamente */
button { font-family: inherit; } /* herda a fonte do pai */

/* Resetar para o valor padrão do navegador */
p { color: initial; }
```

Código 13. Herança de propriedades CSS.

Propriedades herdadas incluem: color, font-family, font-size, font-weight, line-height, text-align, list-style, cursor e visibility. Propriedades não herdadas incluem: margin, padding, border, background, width, height e display. Quando em dúvida, a referência MDN lista a herança de cada propriedade.

6. O Modelo de Caixa (Box Model)

Todo elemento HTML é tratado pelo navegador como uma **caixa retangular**. O modelo de caixa (*box model*) é o conceito mais fundamental do layout CSS: ele descreve exatamente quanto espaço um elemento ocupa na página. Compreender o modelo de caixa é essencial para construir layouts precisos e para depurar problemas de espaçamento — a maioria dos “bugs de layout” tem sua causa raiz aqui.

O modelo de caixa é composto por quatro camadas concêntricas:



Figura 1: As quatro camadas do modelo de caixa do CSS.

- **content**: área onde o conteúdo (texto, imagem) é exibido. Dimensionado por `width` e `height`.
- **padding**: espaço interno entre o conteúdo e a borda. Tem a cor de fundo do elemento — visualmente faz parte do elemento.
- **border**: linha ao redor do padding. Pode ter cor, estilo e espessura. Faz parte do tamanho visual do elemento.
- **margin**: espaço externo entre o elemento e os vizinhos. É transparente e não pertence ao elemento — é o espaço entre os elementos.

```
.card {
  /* Conteúdo */
  width: 300px;
  height: 200px;

  /* Padding: interno (afasta o conteúdo da borda) */
  padding: 16px;           /* todos os lados */
  padding: 8px 16px;       /* vertical horizontal */
  padding: 8px 16px 8px 16px; /* top right bottom left */

  /* Border */
  border: 1px solid #cccccc; /* espessura estilo cor */
  border-radius: 8px;        /* cantos arredondados */

  /* Margin: externo (afasta o elemento dos vizinhos) */
  margin: 0 auto;           /* centraliza horizontalmente (com width definido) */
  margin-bottom: 24px;
}
```

Código 14. Propriedades do box model em CSS.

Um fenômeno importante da margin é o *margin collapsing*: quando dois elementos em bloco adjacentes têm margem inferior e superior respectivamente, as duas margens não se somam — a maior prevalece. Isso é intencional e acontece apenas na direção vertical. Compreender esse comportamento evita confusão ao tentar criar espaçamento entre parágrafos e títulos.

6.1. box-sizing: border-box

Por padrão (content-box), `width` e `height` se referem apenas ao conteúdo. Padding e border são adicionados *por fora*, aumentando o tamanho total do elemento. Isso torna o dimensionamento contraintuitivo: um elemento de `width: 300px` com `padding: 16px` e `border: 1px` ocupa, na verdade, 334px. Com `border-box`, `width` e `height` incluem padding e border — o tamanho definido é o tamanho final.

```
/* content-box (padrão): tamanho real = width + padding + border */
/* Um elemento com width: 300px, padding: 16px, border: 1px
   ocupa efetivamente 300 + 32 + 2 = 334px de largura */

/* border-box: width JÁ inclui padding e border */
/* width: 300px sempre significa 300px de largura total */

/* Aplicar a todos os elementos (recomendado como reset global) */
*, *::before, *::after {
  box-sizing: border-box;
}
```

💡 **Sempre defina box-sizing: border-box globalmente.** A primeira regra de qualquer folha de estilos deve ser o reset de **box-sizing** para **border-box**. Isso torna o dimensionamento previsível: o valor de **width** é sempre o tamanho final do elemento, independentemente de padding e border. Quase todos os *frameworks* CSS modernos (Bootstrap, Tailwind) já fazem isso por padrão.

Para pensar: Abra o DevTools do Chrome (F12), inspecione qualquer elemento da página e localize o painel **Computed** ou o diagrama do box model (geralmente na aba **Styles** ou **Layout**). Identifique os valores de content, padding, border e margin do elemento. Mude o **box-sizing** para **content-box** e depois para **border-box** e observe como o tamanho total se comporta.

7. Unidades de Medida

O CSS oferece unidades absolutas e relativas. Para web, as unidades relativas são geralmente preferíveis por se adaptarem ao contexto do usuário e ao tamanho da tela. A escolha da unidade errada pode tornar uma página inacessível (texto que não escala com as preferências do usuário) ou quebrada em determinados tamanhos de tela.

Unidade	Tipo	Descrição e uso recomendado
px	Absoluta	Pixel. Use para bordas, sombras e valores que não devem escalar.
rem	Relativa	Relativa ao font-size do elemento raiz (html). Padrão: 1rem = 16px. Use para fontes, espaçamentos e layout.
em	Relativa	Relativa ao font-size do elemento pai. Útil para padding/margin proporcional à fonte.
%	Relativa	Porcentagem do elemento pai. Use para larguras fluidas.
vw	Viewport	1% da largura do viewport. Use para layouts em tela cheia.
vh	Viewport	1% da altura do viewport. Use para seções de altura total.
ch	Relativa	Largura do caractere “0” da fonte atual. Útil para largura de colunas de texto.

```
html { font-size: 16px; } /* base: 1rem = 16px */

body {
  font-size: 1rem;        /* 16px escala com preferência do usuário */
  max-width: 65ch;        /* largura ideal para leitura (~65 caracteres) */
  padding: 0 1rem;
}

h1 { font-size: 2rem; } /* 32px */
h2 { font-size: 1.5rem; } /* 24px */
```

Código 15. content-box vs. border-box.

Tabela 3. Principais unidades de medida em CSS.

Código 16. Uso prático das unidades de medida.


```
.hero {
  height: 100vh;      /* ocupa toda a altura da janela */
  width: 100%;
}
```

A diferença entre `rem` e `em` é sutil mas importante em cenários de aninhamento. `em` é relativo ao pai imediato: se um elemento pai tem `font-size: 1.25rem` e o filho usa `padding: 1em`, o padding será `1.25rem`. Se esse filho tiver um neto com `font-size: 0.8em`, o tamanho resulta em $0.8 * 1.25\text{rem} = 1\text{rem}$. Esse efeito composto pode ser confuso. `rem` sempre se refere ao elemento raiz — previsível em qualquer contexto.

💡 **Prefira `rem` para fontes e espaçamentos.** `rem` é relativo ao tamanho de fonte do elemento `<html>`. Se o usuário configurou uma fonte maior no navegador por necessidade de acessibilidade, toda a página escala proporcionalmente. `px` ignora essa preferência. Por isso, use `rem` para `font-size`, `margin`, `padding` e tamanhos de layout. Reserve `px` para bordas e valores que genuinamente não devem escalar.

8. Propriedades de Texto e Cor

8.1. Tipografia

A tipografia é um dos aspectos mais impactantes da apresentação visual. Uma boa tipografia melhora a legibilidade, estabelece hierarquia e comunica a identidade do produto. As propriedades CSS de tipografia controlam todos os aspectos do texto: família, tamanho, peso, estilo, altura da linha e alinhamento.

```
body {
  font-family: 'Inter', Arial, sans-serif; /* pilha de fontes */
  font-size: 1rem;      /* tamanho */
  font-weight: 400;     /* espessura: 100 900, ou normal/bold */
  font-style: normal;   /* normal | italic | oblique */
  line-height: 1.6;     /* altura da linha (sem unidade = relativo) */
  letter-spacing: 0.01em; /* espaçamento entre letras */
  text-align: left;     /* left | center | right | justify */
  text-decoration: none; /* none | underline | line-through */
  text-transform: none; /* none | uppercase | lowercase | capitalize */
}
```

Código 17. Propriedades tipográficas essenciais.

A `font-family` aceita uma **pilha de fontes**: se a primeira não estiver disponível no dispositivo do usuário, o navegador tenta a seguinte, e assim por diante. A última fonte deve sempre ser uma família genérica (`serif`, `sans-serif`, `monospace`) como fallback garantido. A `line-height` sem unidade é o valor mais seguro: significa um múltiplo do `font-size` do elemento, não do pai — evitando problemas de herança. Para texto de corpo, valores entre 1.4 e 1.7 são ideais.

```
<head>
  <!-- Pré-conectar ao servidor de fontes para reduzir latência -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <!-- Carregar a fonte Inter em pesos 400 e 700 -->
  <link
    ↪ href="https://fonts.googleapis.com/css2?family=Inter:wght@400;700&display=swap"
    rel="stylesheet">
</head>
```

Código 18. Carregando fontes externas do Google Fonts.

```
/* Após carregar a fonte no HTML, use-a normalmente no CSS */
body {
  font-family: 'Inter', Arial, sans-serif;
}
```

Código 19. Usando a fonte carregada no CSS.

💡 **Use `display=swap` ao carregar web fonts.** O parâmetro `display=swap` na URL do Google Fonts faz com que o texto seja exibido imediatamente com a fonte de fallback enquanto a fonte personalizada carrega. Sem isso (`display=block`, o

padrão), o texto fica invisível durante o download — o chamado *FOIT* (*Flash of Invisible Text*). *swap* produz um *FOUT* (*Flash of Unstyled Text*), que é visualmente mais tolerável.

8.2. Cores

O CSS aceita cores em vários formatos. A escolha do formato afeta a legibilidade e a manutenibilidade do código: hexadecimal é compacto e amplamente usado; HSL é mais intuitivo para ajustar matiz e luminosidade; variáveis CSS são a forma mais organizada para projetos com um sistema de cores.

```
/* Nome da cor (147 nomes predefinidos) */
color: red;
color: cornflowerblue;

/* Hexadecimal: #RRGGBB ou abreviado #RGB */
color: #1f4e79;
color: #fff; /* equivale a #ffffff */

/* RGB e RGBA (A = opacidade, 01) */
color: rgb(31, 78, 121);
color: rgba(31, 78, 121, 0.5); /* 50% transparente */

/* HSL: matiz (0360), saturação, luminosidade */
color: hsl(210, 60%, 30%);

/* Variáveis CSS (Custom Properties) */
:root { --cor-primaria: #1f4e79; }
h1 { color: var(--cor-primaria); }
```

Código 20. Formatos de cor em CSS.

O formato HSL tem uma vantagem prática: é fácil criar variações de uma cor apenas alterando a luminosidade. Por exemplo, `hsl(210, 60%, 30%)` (azul escuro), `hsl(210, 60%, 50%)` (médio) e `hsl(210, 60%, 90%)` (azul muito claro para fundos) são claramente relacionadas — algo difícil de perceber a partir dos valores hexadecimais correspondentes.

 **Use variáveis CSS para consistência de cores.** Definir as cores do projeto como variáveis CSS no `:root` é a forma mais organizada de manter um sistema de cores consistente. Se a cor primária mudar, você altera em um único lugar e todas as regras que usam `var(--cor-primaria)` são atualizadas automaticamente — princípio DRY aplicado ao CSS.

8.3. Dimensões: *width*, *height* e *overflow*

```
.elemento {
  /* Largura e altura fixas */
  width: 400px;
  height: 200px;

  /* Limites mínimos e máximos (mais flexível que valores fixos) */
  min-width: 200px; max-width: 800px;
  min-height: 100px; max-height: 400px;

  /* overflow: o que fazer quando o conteúdo excede a caixa */
  overflow: visible; /* padrão conteúdo transborda */
  overflow: hidden; /* corta o que ultrapassar */
  overflow: scroll; /* sempre exibe barra de rolagem */
  overflow: auto; /* barra de rolagem só quando necessário */

  /* Controle por eixo */
  overflow-x: hidden; /* corta horizontalmente */
  overflow-y: auto; /* rola verticalmente se necessário */
}
```

Código 21. Largura, altura e comportamento do conteúdo excedente.

Prefira `max-width` a `width` fixo para layouts fluidos: `max-width: 800px` significa que o elemento nunca excede 800px, mas pode ser menor em telas estreitas. `width: 800px` força o elemento a ter exatamente 800px mesmo em telas de 375px, causando rolagem horizontal indesejada.

8.4. Display e visibilidade

A propriedade `display` é uma das mais importantes do CSS: ela define como um elemento participa do fluxo de layout da página. Compreender a diferença entre os valores principais é essencial antes de aprender Flexbox e Grid, pois ambos são valores de `display`.

```
/* block: ocupa toda a largura disponível, quebra linha antes e depois */
/* Padrão de: div, p, h1h6, section, article, header, footer */
.bloco { display: block; }

/* inline: flui junto com o texto, não quebra linha */
/* Padrão de: span, a, strong, em, img */
.em-linha { display: inline; }

/* inline-block: flui como inline, mas aceita width/height como block */
.botao { display: inline-block; width: 120px; text-align: center; }

/* flex e grid: ativam os sistemas de layout (seções 7 e 8) */
.container-flex { display: flex; }
.container-grid { display: grid; }

/* none: remove o elemento do fluxo (invisível e sem espaço) */
.oculto { display: none; }
```

Código 22. Valores essenciais de `display`.

```
/* visibility: hidden invisível, mas ainda ocupa espaço no layout */
.placeholder { visibility: hidden; }

/* opacity: transparência (0 = invisível, 1 = opaco) */
/* 0 elemento ainda ocupa espaço e recebe eventos de mouse */
.semitransparente { opacity: 0.5; }

/* pointer-events: none ignora cliques e hover (útil com opacity) */
.nao-interativo { opacity: 0.4; pointer-events: none; }
```

Código 23. Visibilidade sem remover do fluxo.

A diferença entre `display: none` e `visibility: hidden` é importante: `display: none` remove completamente o elemento do fluxo — o espaço que ele ocuparia é recalculado, como se o elemento não existisse. `visibility: hidden` apenas o torna invisível, mas o espaço é preservado. Use `display: none` para mostrar/esconder conteúdo; use `visibility: hidden` quando precisar reservar o espaço.

8.5. Background

```
.elemento {
  /* Cor de fundo */
  background-color: #f0f4f8;

  /* Imagem de fundo */
  background-image: url('/img/textura.png');
  background-repeat: no-repeat; /* repeat | repeat-x | repeat-y */
  background-size: cover; /* cover | contain | 100px 200px */
  background-position: center top; /* horizontal vertical */

  /* Gradiente linear (sem imagem externa) */
  background-image: linear-gradient(135deg, #1f4e79, #2e75b6);

  /* Atalho: background reúne todas as sub-propriedades */
  background: #f0f4f8 url('/img/logo.png') no-repeat center / 80px;
}
```

Código 24. Propriedades de fundo: cor, imagem e gradiente.

8.6. Cursor e transições

```
/* cursor: indica ao usuário o tipo de interação disponível */
.link { cursor: pointer; } /* mãozinha elemento clicável */
.texto { cursor: text; } /* cursor de texto */
.espera { cursor: wait; } /* ampulheta processando */
.mover { cursor: move; } /* setas em cruz arrastável */
.proibido { cursor: not-allowed; } /* círculo cortado desabilitado */

/* user-select: controla se o texto pode ser selecionado */
.no-select { user-select: none; } /* impede seleção (ex.: botões) */

/* transition: anima a mudança de propriedades */
.botoao {
  background-color: #2e75b6;
  transition: background-color 0.2s ease;
}
.botoao:hover { background-color: #1f4e79; }
```

Código 25. Cursor e propriedades de interação.

A propriedade `transition` faz o CSS animar automaticamente a mudança de um valor para outro. A sintaxe é `transition: propriedade duração timing-function`. O valor `ease` começa rápido e desacelera; `linear` mantém velocidade constante; `ease-in-out` começa e termina devagar. Para múltiplas propriedades, separe com vírgula: `transition: background-color 0.2s ease, transform 0.1s ease`.

💡 Use `cursor: pointer` em qualquer elemento clicável. Por padrão, apenas tags `<a>` e `<button>` exibem a mãozinha ao passar o mouse. Se você tornar um `<div>` ou `<span>` clicável via JavaScript, adicione `cursor: pointer` para comunicar visualmente a interatividade. O mesmo vale para elementos com `role="button"` ou `tabindex`.

9. Layout com Flexbox

O Flexbox (*Flexible Box Layout*) é o sistema de layout do CSS projetado para distribuir itens em **uma dimensão** — uma linha ou uma coluna. Introduzido em 2012, o Flexbox resolveu problemas de layout que antes exigiam hacks com floats e posicionamento negativo: centralização vertical, distribuição uniforme de espaço, e reordenação de elementos independentemente da ordem no HTML.

9.1. Conceitos do Flexbox

Para usar Flexbox, aplica-se `display: flex` ao **container**. Os filhos diretos tornam-se **itens flex** automaticamente. O container controla a distribuição e o alinhamento dos itens; cada item pode ajustar seu próprio crescimento e encolhimento.

```
.container {
  display: flex;

  /* Direção dos itens: row (padrão) | column | row-reverse */
  flex-direction: row;

  /* Quebra de linha quando não cabem: nowrap | wrap */
  flex-wrap: wrap;

  /* Alinhamento no eixo principal (horizontal em row) */
  justify-content: space-between;
  /* flex-start | flex-end | center | space-between | space-around */

  /* Alinhamento no eixo cruzado (vertical em row) */
  align-items: center;
  /* stretch | flex-start | flex-end | center | baseline */

  /* Espaçamento entre itens (substitui margin em muitos casos) */
  gap: 1rem;
}
```

Código 26. Propriedades do container flex.

Os dois eixos do Flexbox são o **eixo principal** (na direção do `flex-direction`) e o **eixo cruzado** (perpendicular ao principal). `justify-content` controla o eixo principal; `align-items` controla o eixo cruzado. Essa distinção é a chave para não confundir as propriedades: quando `flex-direction: column`, o eixo principal é vertical e os papéis se invertem.

```
.item {
  /* flex: crescimento encolhimento base */
  flex: 1;          /* cresce e encolhe igualmente, base: auto */
  flex: 0 0 200px; /* largura fixa de 200px, não cresce nem encolhe */

  /* Alinhamento individual (sobrescreve align-items do container) */
  align-self: flex-end;

  /* Ordem de exibição (independente da ordem no HTML) */
  order: 2;
}

/* Barra de navegação: logo à esquerda, links à direita */
nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem;
}

/* Centralizar um elemento horizontal e verticalmente */
.centralizado {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

/* Cards em linha que quebram quando necessário */
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1.5rem;
}
.cards .card { flex: 1 1 280px; } /* mínimo 280px, cresce */
```

Código 27. Propriedades dos itens flex.

Código 28. Padrões comuns com Flexbox.

Para pensar e implementar: Usando apenas Flexbox (sem Grid), construa um layout de página com header, área de conteúdo e footer. Dentro da área de conteúdo, crie uma linha com três cards de largura igual. Faça com que os cards se empilhem verticalmente em telas menores que 600px, usando uma media query que altera o `flex-direction` para `column`. Qual propriedade do item flex garante que os três cards sempre ocupem o mesmo espaço?

10. Layout com Grid

O CSS Grid é o sistema de layout projetado para **duas dimensões** — linhas e colunas ao mesmo tempo. Enquanto o Flexbox distribui itens em um único eixo, o Grid permite definir explicitamente onde cada elemento aparece na grade — tanto na horizontal quanto na vertical. É ideal para o layout geral da página, galerias de imagens e qualquer estrutura tabular complexa.

10.1. Conceitos do Grid

Assim como no Flexbox, aplica-se `display: grid` ao container. A diferença fundamental é que, com Grid, você define a estrutura da grade *antes* de posicionar os itens — o que permite um controle preciso sobre linhas e colunas. A unidade `fr` (fração) é exclusiva do Grid e representa uma fração do espaço disponível após subtrair espaços fixos.

```
.grid {
  display: grid;

  /* 3 colunas de tamanhos fixos */
  grid-template-columns: 200px 1fr 1fr;

  /* fr = fração do espaço disponível */
  /* 1fr 1fr 1fr = três colunas iguais */

  /* Atalho: repeat(n, tamanho) */
  grid-template-columns: repeat(3, 1fr);

  /* Linhas automáticas com altura mínima */
  grid-auto-rows: minmax(100px, auto);

  gap: 1rem; /* espaço entre células */
}
```

Código 29. Definindo colunas e linhas no Grid.

```
/* Layout clássico de página */
.pagina {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main "
    "footer footer";
  min-height: 100vh;
}

header { grid-area: header; }
.sidebar { grid-area: sidebar; }
main { grid-area: main; }
footer { grid-area: footer; }
```

Código 30. Posicionamento de itens no Grid.

O recurso de `grid-template-areas` é particularmente poderoso: permite visualizar o layout diretamente no CSS, com uma representação textual da grade. Cada string representa uma linha; cada palavra, uma célula. O ponto `.` representa uma célula vazia. Uma área que se repete em múltiplas células indica que o item ocupa aquele espaço.

```
/* Galeria responsiva sem media query:
   colunas de no mínimo 250px, preenchendo o espaço */
.galeria {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 1rem;
}

/* auto-fill: cria quantas colunas couberem
   minmax(250px, 1fr): cada coluna tem no mínimo 250px */
```

Código 31. Grid com número automático de colunas.

O padrão `repeat(auto-fill, minmax(250px, 1fr))` é um dos mais úteis do CSS moderno: cria uma galeria responsiva sem nenhuma *media query*. Em uma tela de 1200px, cabem quatro colunas de 300px cada; em 800px, três colunas; em 500px, duas. O layout se adapta automaticamente.

Flexbox ou Grid? Use **Flexbox** quando o layout é *unidimensional*: uma linha de botões, uma barra de navegação, uma linha de cards. Use **Grid** quando o layout é *bidimensional*: a estrutura geral da página, uma galeria com linhas e colunas. Na prática, os dois se complementam: Grid define o layout macro, Flexbox organiza os componentes internos.

11. Responsividade e Media Queries

Um layout responsivo se adapta a diferentes tamanhos de tela — celulares, tablets e desktops — sem precisar de páginas separadas. A ferramenta principal são as **media queries**: blocos CSS que só se aplicam quando uma condição é satisfeita. Media

queries podem verificar a largura do viewport, a orientação da tela, a resolução e até preferências do sistema operacional, como o modo escuro.

A abordagem **mobile-first** consiste em escrever primeiro os estilos para telas pequenas e usar `@media (min-width: ...)` para adicionar complexidade progressivamente. É o inverso do *desktop-first*, que escreve para telas grandes e usa `max-width` para simplificar. Mobile-first produz CSS mais enxuto e reflete a realidade do consumo da web: mais de 60% do tráfego global vem de dispositivos móveis.

```
/* Estilos base: mobile-first (telas pequenas) */
.cards { flex-direction: column; }
nav { display: none; } /* menu escondido em mobile */

/* Telas com 768px ou mais (tablet) */
@media (min-width: 768px) {
  .cards { flex-direction: row; flex-wrap: wrap; }
  nav { display: flex; }
}

/* Telas com 1200px ou mais (desktop) */
@media (min-width: 1200px) {
  body { font-size: 1.125rem; }
  .cards { gap: 2rem; }
}

/* Preferência do sistema: modo escuro */
@media (prefers-color-scheme: dark) {
  body { background: #121212; color: #e0e0e0; }
}
```

Código 32. Sintaxe e uso de media queries.

Além das media queries de largura, o CSS moderno oferece media queries de *preferências do usuário*: `prefers-color-scheme` para o modo claro/escuro, `prefers-reduced-motion` para usuários sensíveis a animações, e `prefers-contrast` para maior contraste. Implementar suporte a essas preferências é uma forma simples e eficaz de melhorar a acessibilidade da aplicação.

```
/* Desativa animações para usuários que as desabilitaram no sistema */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

Código 33. Media query para preferências de acessibilidade.

 **Adote a abordagem mobile-first.** Escreva os estilos base pensando na tela menor (mobile) e use `@media (min-width: ...)` para adicionar complexidade progressivamente. Essa abordagem produz CSS mais enxuto do que o inverso (*desktop-first* com `max-width`). Além disso, o Google usa a versão mobile para indexação (*mobile-first indexing*) — o que torna a abordagem relevante também para SEO.

12. Boas Práticas e Organização

12.1. Nomenclatura com BEM

BEM (*Block, Element, Modifier*) é uma convenção de nomenclatura de classes que torna o CSS mais legível, modular e evita conflitos de nomes em projetos grandes. O BEM divide as classes em três categorias: blocos independentes, elementos que pertencem a um bloco, e modificadores que representam variações. Um projeto que adota BEM consistentemente raramente precisa de seletores com alta especificidade.

```
/* Bloco: componente independente */
.card { ... }

/* Elemento: parte do bloco (separado por __) */
.card__titulo { font-size: 1.25rem; }
```

Código 34. Convenção BEM de nomenclatura de classes.

```
.card__imagem { width: 100%; }
.card__rodape { border-top: 1px solid #eee; }

/* Modificador: variação do bloco ou elemento (separado por --) */
.card--destaque { border: 2px solid #2e75b6; }
.card--destaque .card__titulo { color: #2e75b6; }

<article class="card card--destaque">
  
  <h3 class="card__titulo">Handout 01</h3>
  <footer class="card__rodape">
    <a class="card__link" href="handout01.pdf">Baixar</a>
  </footer>
</article>
```

Código 35. BEM aplicado ao HTML.

12.2. Organização do arquivo CSS

Uma folha de estilos bem organizada segue uma ordem lógica: reset e configurações globais primeiro, depois variáveis, estilos base, layout, componentes e, por último, as media queries. Essa organização reflete a cascata natural do CSS e torna mais previsível qual regra prevalece em caso de conflito.

```
/* 1. Reset e configurações globais */
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; padding: 0; }

/* 2. Variáveis CSS (Design Tokens) */
:root {
  --cor-primaria: #1f4e79;
  --cor-texto: #333333;
  --fonte-base: 'Inter', Arial, sans-serif;
  --espaco-base: 1rem;
}

/* 3. Estilos base (elementos HTML) */
body { font-family: var(--fonte-base); color: var(--cor-texto); }
h1, h2, h3 { color: var(--cor-primaria); }

/* 4. Layout (header, nav, main, footer) */
/* 5. Componentes (cards, botões, formulários) */
/* 6. Utilitários (classes auxiliares) */
/* 7. Media queries */
```

Código 36. Estrutura recomendada de um arquivo CSS.

⚠ Evite seletores muito específicos e aninhamento profundo. Seletores como `header nav ul li a:hover` são difíceis de reutilizar e criam especificidade desnecessariamente alta. Prefira classes únicas e descritivas: `.nav__link:hover`. Em geral, limite o aninhamento de seletores a no máximo dois níveis.

13. Servindo CSS pelo Back-end com Express

No handout de HTML vimos que o `express.static` serve toda a pasta `public` — incluindo arquivos CSS, JavaScript e imagens. Para o CSS, essa é a forma correta e suficiente na maioria dos projetos: o arquivo `.css` fica em `public/css/` e o navegador o busca automaticamente ao encontrar um `<link rel="stylesheet">` no HTML.

13.1. O caminho do CSS na pasta estática

```
projeto/
+-- src/
|   +-- app.js
+-- public/
|   +-- index.html
|   +-- css/
|       +-- estilo.css      <- acessado em GET /css/estilo.css
```

Código 37. Estrutura de pastas com CSS em `public/css/`.


```

| | +-- componentes.css
| +-- js/
| | +-- script.js
| +-- img/
|     +-- logo.png
+-- package.json

const express = require('express');
const path    = require('path');
const app     = express();

// Um único middleware serve HTML, CSS, JS e imagens
app.use(express.static(path.join(__dirname, '..', 'public')));

// GET /css/estilo.css      -> public/css/estilo.css
// GET /css/componentes.css -> public/css/componentes.css
// GET /index.html         -> public/index.html

app.listen(3000, () => console.log('Servidor em http://localhost:3000'));

```

Código 38. `express.static` serve CSS junto com HTML e imagens.

No HTML, o `<link>` aponta para o caminho relativo à raiz da pasta pública:

```

<head>
  <meta charset="UTF-8">
  <title>Página da disciplina</title>
  <!-- /css/estilo.css resolve para public/css/estilo.css no servidor -->
  <link rel="stylesheet" href="/css/estilo.css">
</head>

```

Código 39. Referenciando o CSS no HTML servido pelo Express.

💡 Use sempre caminhos absolutos a partir da raiz no href. Prefira `href="/css/estilo.css"` (com a barra inicial) a `href="css/estilo.css"` (caminho relativo). Caminhos relativos dependem da URL atual — uma página em `/admin/painel` buscaria `/admin/css/estilo.css`, que não existe. O caminho absoluto a partir da raiz funciona em qualquer página, independentemente de quantos níveis de rota existam.

13.2. Múltiplas folhas de estilo e ordem de carregamento

É comum separar os estilos em arquivos com responsabilidades distintas. O navegador os carrega na ordem em que aparecem no HTML — regras em arquivos posteriores sobrescrevem as anteriores em caso de conflito (princípio da cascata). Essa ordem deve refletir a organização interna do CSS descrita na seção anterior.

```

<head>
  <!-- 1. Reset global sempre primeiro -->
  <link rel="stylesheet" href="/css/reset.css">
  <!-- 2. Variáveis e design tokens -->
  <link rel="stylesheet" href="/css/variaveis.css">
  <!-- 3. Estilos base (tipografia, cores, layout) -->
  <link rel="stylesheet" href="/css/base.css">
  <!-- 4. Componentes (cards, botões, formulários) -->
  <link rel="stylesheet" href="/css/componentes.css">
  <!-- 5. Estilos específicos da página sempre por último -->
  <link rel="stylesheet" href="/css/pagina-alunos.css">
</head>

```

Código 40. Múltiplos arquivos CSS com responsabilidades distintas.

13.3. Servindo CSS gerado dinamicamente

Em casos avançados — como gerar CSS com variáveis vindas do banco (temas por usuário, cores configuráveis) — é possível criar uma rota Express que produz CSS dinamicamente e o entrega com o *Content-Type* correto:

```

const express = require('express');
const router  = express.Router();
const db      = require('../models');

```

Código 41. Rota que gera e entrega CSS dinâmico.

```
// GET /tema/estilo.css?usuario=42
router.get('/estilo.css', async (req, res) => {
  const config = await db.ConfiguracaoTema.findOne({
    where: { id_usuario: req.query.usuario }
  });

  const corPrimaria = config?.cor_primaria ?? '#1f4e79';
  const corTexto = config?.cor_texto ?? '#333333';

  res.set('Content-Type', 'text/css');
  res.send(`
    :root {
      --cor-primaria: ${corPrimaria};
      --cor-texto: ${corTexto};
    }
  `);
});

module.exports = router;
```

⚠ Nunca interpole dados do usuário diretamente em CSS. O código acima usa valores vindos do banco — já validados e controlados pela aplicação. Nunca interpole `req.query`, `req.body` ou qualquer outro dado diretamente fornecido pelo usuário em CSS sem validação estrita. Um valor como `color: red} body { display:none` pode encerrar o bloco CSS e injetar regras arbitrárias — o equivalente do XSS no contexto de folhas de estilo. Valide o formato dos valores (ex.: aceitar apenas `#rrggbb` com `regex`) antes de interpolar.

```
<head>
  <!-- CSS estático: base e componentes -->
  <link rel="stylesheet" href="/css/base.css">
  <!-- CSS dinâmico: tema personalizado por usuário -->
  <link rel="stylesheet"
    href="/tema/estilo.css?usuario=<%= usuario.id %>">
</head>
```

Código 42. Referenciando o CSS dinâmico no template EJS.

CSS dinâmico vs. variáveis CSS no cliente Gerar CSS no servidor é uma das abordagens para temas dinâmicos, mas não a única. Uma alternativa comum é servir um único CSS estático com variáveis CSS (`--cor-primaria: #1f4e79`) e sobrescrevê-las via JavaScript no cliente: `document.documentElement.style.setProperty('--cor-primaria', cor)`. Essa abordagem não requer nenhuma rota extra, mas as preferências precisam ser armazenadas e lidas no *front-end* (ex.: `localStorage` ou um endpoint de perfil). Para temas complexos ou renderizados no servidor com EJS, a rota de CSS dinâmico é mais adequada.

14. Para Pensar e Implementar

Exercício 1 — Regras e seletores: Crie um arquivo `estilo.css` e aplique-o à página HTML criada no handout anterior. Escreva regras para: (1) definir a família de fontes de todo o `body`; (2) colorir todos os `h1` e `h2` com a cor primária da UFFS; (3) remover o sublinhado dos links dentro do `nav` e alterá-lo ao passar o mouse (`:hover`). Identifique o seletor usado em cada regra e calcule sua especificidade.

Exercício 2 — Box model: Crie uma `<div class="card">` com um título, um parágrafo e um link. Usando apenas CSS, adicione: `padding` interno de `1.5rem`, `border` de `1px solid`, `border-radius` de `8px`, `margin-bottom` de `2rem` e uma `box-shadow` suave. Inspecione o elemento no DevTools do Chrome e observe cada

camada do box model destacada no painel. Compare o comportamento com `box-sizing: content-box` e `border-box`.

Exercício 3 — Flexbox: Usando Flexbox, construa uma barra de navegação com o logo à esquerda e três links à direita, alinhados verticalmente ao centro. Em seguida, crie uma linha de três cards com espaçamento uniforme entre eles. Use `flex-wrap: wrap` para que os cards quebrem para a próxima linha em telas estreitas.

Exercício 4 — Grid e layout de página: Usando `grid-template-areas`, construa o layout completo da página do exercício anterior com header, navegação lateral (sidebar), conteúdo principal e footer. Faça com que a sidebar desapareça em telas menores que 768px, usando uma media query que altera o grid para uma única coluna.

Exercício 5 — Responsividade mobile-first: Refatore os estilos dos exercícios anteriores adotando a abordagem *mobile-first*: estilos base para mobile, com `@media (min-width: 768px)` para tablet e `@media (min-width: 1200px)` para desktop. Teste redimensionando a janela do navegador ou usando o modo de dispositivo do DevTools (Ctrl+Shift+M no Chrome).

Exercício 6 — Variáveis CSS e modo escuro: Defina as cores e fontes do seu projeto como variáveis CSS no `:root`. Adicione suporte a modo escuro com `@media (prefers-color-scheme: dark)`, sobrescrevendo as variáveis de cor. Abra o DevTools, vá em *Rendering* e ative “Emulate CSS media feature prefers-color-scheme: dark” para testar sem mudar as configurações do sistema.

Resumo dos Pontos-Chave

- **CSS controla a apresentação:** separado do HTML por razões de manutenção e reutilização. Um arquivo CSS externo serve múltiplas páginas e é armazenado em cache pelo navegador.
- **Regra CSS:** seletor + bloco de declarações com pares propriedade/valor. Sempre use ponto-e-vírgula após cada declaração.
- **Seletores:** prefira classes para estilização (especificidade moderada e reutilizável). Reserve IDs para JavaScript e âncoras. Evite seletores com mais de dois níveis de aninhamento.
- **Cascata e especificidade:** inline > ID > classe > elemento. Regras de mesma especificidade: a última vence. Evite `!important` — ele mascara problemas de arquitetura.
- **Herança:** propriedades como `color` e `font-family` são herdadas pelos filhos. `border`, `padding` e `margin` não são.
- **Box model:** todo elemento é uma caixa com content, padding, border e margin. Sempre use `box-sizing: border-box` globalmente para que `width` represente o tamanho total.
- **Unidades:** use `rem` para fontes e espaçamentos (respeita preferências de acessibilidade do usuário). Use `px` para bordas. Use `%` e `vw/vh` para layouts fluidos.
- **Flexbox:** layout unidimensional. O container controla direção, quebra e alinhamento. Os itens controlam crescimento e ordem.
- **Grid:** layout bidimensional. Define linhas e colunas explicitamente. `grid-template-areas` permite visualizar o layout no CSS. O padrão `repeat(auto-fill, minmax(...))` cria galerias responsivas sem media query.

- **Responsividade:** adote *mobile-first*. Estilos base para mobile; @media (min-width: ...) para telas maiores. prefers-color-scheme e prefers-reduced-motion para preferências do sistema.
 - **BEM:** convenção de nomenclatura bloco__elemento-modificador que torna classes autodescritivas e evita conflitos de especificidade.
 - **Variáveis CSS:** use :root { --nome: valor } para design tokens. Facilita manutenção e temas dinâmicos.
 - **Express + CSS:** express.static serve arquivos CSS em public/css/. Use caminhos absolutos (/css/estilo.css) no href. Para temas dinâmicos, crie uma rota que entrega CSS com Content-Type: text/css — mas nunca interpole dados do usuário sem validação.
-

Referências e Leituras Complementares

- **MDN Web Docs — CSS Reference:** <https://developer.mozilla.org/pt-BR/docs/Web/CSS>
- **MDN — Aprendendo CSS:** <https://developer.mozilla.org/pt-BR/docs/Learn/CSS>
- **CSS Tricks — A Complete Guide to Flexbox:** <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- **CSS Tricks — A Complete Guide to CSS Grid:** <https://css-tricks.com/snippets/css/complete-guide-grid/>
- **web.dev — Learn CSS:** <https://web.dev/learn/css/> — curso moderno e aprofundado do Google
- **CSS Zen Garden:** <https://csszengarden.com> — mesmo HTML, dezenas de designs diferentes
- **Specificity Calculator:** <https://specificity.keegan.st/> — calcula a especificidade de qualquer seletor
- **BEM Methodology:** <https://getbem.com/> — documentação oficial da convenção BEM
- **Can I Use:** <https://caniuse.com/> — compatibilidade de propriedades CSS entre navegadores
- **Google Fonts:** <https://fonts.google.com/> — biblioteca gratuita de fontes web
- **MDN — Using CSS custom properties:** https://developer.mozilla.org/pt-BR/docs/Web/CSS/Using_CSS_custom_properties